

5-4 · DataFrame과 SparkSQL 가공 · 실습 1.5H

DataFrame API로 데이터 가공

조인·파생·필터·집계를 체인으로 — 중간마다 확인

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

① orders·products 두 DataFrame 을 만들고 order_date 를 date 로 바꾼다

② join → withColumn → filter → groupBy 를 체인으로 잇는다

③ 조인 전후 행 수와 오른쪽 키 고유성을 직접 확인한다

④ 윈도우 함수로 날짜별 1위를 뽑고 explain() 에서 조인 방식을 읽는다

5-3 셀 옵션 그대로 PySpark 를 띄웁니다 — 옵션이 길면 스크립트로 저장해 두세요

일곱 단계 — 표 두 개에서 실행 계획까지

3주차 Pandas 에서 한 일을 분산 환경에서 그대로 — 문법이 조금 다를 뿐 생각은 같습니다

1. 데이터

orders 5행
products 3행

2. 타입

to_date

3·4. 조인·필터

amount 파생
10만 이상

5·6. 집계·윈도

groupBy
row_number

7. 계획

explain()

표 두 개를 만들고 날짜 타입을 바로잡는다

- 1 orders · products 를 createDataFrame 으로**
orders 5행(order_id·product_id·qty·order_date) · products 3행(product_id·name·price)
- 2 show() · printSchema() 로 확인**
order_date 가 string — 3-1 에서 겪은 것과 같은 상황
- 3 F.to_date("order_date") 로 바꾼다**
withColumn 으로 덮어쓰고 printSchema() 에 date 가 보이면 통과

먼저 한 줄

**functions
as F**

| from pyspark.sql import functions as F — 날짜는 반드시 변환 후 사용(3주차)

조인 · 파생 · 필터 · 집계 — 핵심 줄

조인	<code>orders.join(products, on="product_id", how="inner")</code>
파생	<code>withColumn("amount", F.col("qty") * F.col("price"))</code>
필터	<code>filter(F.col("amount") >= 100000)</code>
복합 조건	(조건1) & (조건2) — 괄호 필수, Pandas 와 같다
집계	<code>groupBy("name").agg(F.sum("amount").alias("total_amount"))</code>
정렬	<code>orderBy(F.desc("total_amount"))</code>

여러 집계는 `agg` 안에 `F.count("*")` · `F.sum` · `F.avg` · `F.max` 를 나란히 — `alias` 로 이름을 정한다(3-2 명명 집계)

조인 전후 행 수를 센다 — 5행 → 5행

1 `print(orders.count(), joined.count())`
조인 전후 모두 5행 — products 의 product_id 가 고유하므로

2 행이 늘었다면 오른쪽 키 중복
3-2 에서 본 사고 — 합계가 부풀려진다

3 `select("product_id").distinct().count()`
products.count() 와 같으면 안전 — validate 대신 이 한 줄

두 숫자

3 = 3

이면 안전

조인 키를 조건식으로 주면 — ambiguous

이론의 「키를 문자열로」가 왜 규칙인지 — 오류를 한 번 직접 봅니다

1. 조건식 조인	2. 컬럼 선택	3. 오류	4. 처방	5. 한 번 더
<code>a["k"] == b["k"]</code> 형태로	<code>select("product_id")</code>	Reference ... is ambiguous	<code>on="product_id"</code> 문자열로	<code>joined 에 amount</code> → cannot resolve

두 오류 — ambiguous 와 cannot resolve

Reference 'product_id' is ambiguous

양쪽에 같은 이름의 컬럼이 남아 있다

- 원인: 조건식 조인은 두 컬럼을 다 남긴다
- 처방 1: on="product_id" 문자열로
- 처방 2: alias("o").alias("p") 로 출처 명시

VS

cannot resolve 'amount'

그 시점의 DataFrame 에는 없는 컬럼

- 원인: 오타 또는 가공 전 변수(joined) 참조
- 처방: 직전 DataFrame 에 printSchema()
- with_amount 에서 다시 filter

집계 결과 · 윈도우 · 실행 계획

1

sales.show() 순서 확인

monitor 930,000 → keyboard 315,000 → mouse 69,000 이면 집계가 맞다

2

Window.partitionBy("order_date")

F.row_number().over(w) 로 rn 을 만들고 filter("rn = 1") 이 날짜별 1위

3

sales.explain()

products 가 작아 BroadcastHashJoin · groupBy 때문에 Exchange 한 번

1위 행 수

날짜 수

만큼

이 1.5시간이 끝날 때 확인할 것

- 1 orders 5행 · products 3행이 스키마와 함께 출력된다
- 2 printSchema() 에서 order_date 가 date 다
- 3 조인 전후 행 수가 $5 = 5 \cdot \text{products 키 고유성 } 3 = 3$
- 4 10만 이상 필터에 P001 5개 225,000 · P003 3개 930,000 이 남는다
- 5 상품별 합계가 monitor · keyboard · mouse 순이다
- 6 explain() 에 BroadcastHashJoin 과 Exchange 가 보인다

5-4 · 실습 1.5H 마무리

다음은 SparkSQL 실습 (1H)

같은 가공을 SQL 로 다시 쓰고, 두 방식의 explain() 을 대조한 뒤 Iceberg 테이블로 저장합니다

✓ createOrReplaceTempView 로 임시 뷰

✓ SQL 로 같은 집계 — 결과 대조

✓ explain() 둘을 나란히

✓ writeTo("lake.db.product_sales")

가져갈 것

셀 그대로 · sales · with_amount 변수